

✓ Generative AI & Transformers Discovery

Overview

Generative AI & Transformers Pipeline – Text Analysis and Generation (Discovery-to-Action Strategy)

✓ Discovery Phase

```
!pip install transformers torch --quiet
```

```
from transformers import pipeline  
import torch
```

```
print("Environment ready. Torch version:", torch.__version__)
```

```
Environment ready. Torch version: 2.11.0+cpu
```

```
from transformers import pipeline  
import torch
```

```
print("Environment ready. Torch version:", torch.__version__)
```

```
# Initialize pipelines
```

```
sentiment_pipeline = pipeline("sentiment-analysis")
```

```
generator = pipeline("text-generation", model="gpt2")
```

```
summarizer = pipeline("text-generation", model="sshleifer/distilbart-cnn-12-6")
```

```
print("Pipelines initialized successfully!")
```

```
[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased
Using a pipeline without specifying a model name and revision in production is not recommended
Environment ready. Torch version: 2.11.0+cpu
```

```
Loading weights: 100% 104/104 [00:00<00:00, 1973.67it/s]
```

```
Loading weights: 100% 148/148 [00:00<00:00, 3495.23it/s]
```

```
Loading weights: 100% 160/160 [00:00<00:00, 4541.59it/s]
```

```
[transformers] BartForCausalLM LOAD REPORT from: sshleifer/distilbart-cnn-12-6
```

Key	Status		
-----+-----+-----			
model.encoder.layers.{0...11}.final_layer_norm.weight	UNEXPECTED		
model.encoder.layers.{0...11}.fc1.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.k_proj.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.out_proj.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn_layer_norm.bias	UNEXPECTED		
model.encoder.layers.{0...11}.fc1.bias	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.out_proj.bias	UNEXPECTED		
model.shared.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.v_proj.bias	UNEXPECTED		
model.encoder.layers.{0...11}.fc2.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.q_proj.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.k_proj.bias	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.v_proj.weight	UNEXPECTED		
final_logits_bias	UNEXPECTED		
model.encoder.layers.{0...11}.fc2.bias	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn_layer_norm.weight	UNEXPECTED		
model.encoder.layers.{0...11}.self_attn.q_proj.bias	UNEXPECTED		
model.encoder.layers.{0...11}.final_layer_norm.bias	UNEXPECTED		
model.encoder.layer_norm_embedding.weight	UNEXPECTED		
model.encoder.layer_norm_embedding.bias	UNEXPECTED		
model.encoder.embed_positions.weight	UNEXPECTED		
model.encoder.embed_tokens.weight	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if Pipelines initialized successfully!

✓ Debugging the Summarization Pipeline Output

A `KeyError` often means you are trying to access a key that isn't present in the dictionary returned by the pipeline. Let's inspect the full output of the `summarizer` pipeline to see its exact structure.

```
# Use a small input to inspect the summarization pipeline's output structure
# This helps identify the correct keys to access, preventing KeyError.
```

```
debug_article = """
```

```
Artificial intelligence (AI) is intelligence demonstrated by machines, in contrast to
In computer science, AI research is defined as the study of "intelligent agents": any
"""
```

```
# Run the summarizer with a short text and display the full output
full_summary_output = summarizer(debug_article, max_length=30, min_length=10, do_sample=False)

print("Full output of the summarization pipeline:")
display(full_summary_output)

# You can then use the correct key, for example:
# if isinstance(full_summary_output, list) and full_summary_output:
#     if 'summary_text' in full_summary_output[0]:
#         print(f"Accessing 'summary_text': {full_summary_output[0]['summary_text']}")
#     else:
#         print("Key 'summary_text' not found in the first element of the output.")
# else:
#     print("Unexpected output format from the summarizer.")
```

```
[transformers] Passing `generation_config` together with generation-related argument
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer RobertaTokenizer
Full output of the summarization pipeline:
[{'generated_text': '\nArtificial intelligence (AI) is intelligence demonstrated by machines, in contrast to the natural intelligence displayed by humans and other animals.\nIn computer science, AI research is defined as the study of "intelligent agents": any device that perceives its environment and takes actions that maximize its chance of successfully achieving its goals.\n'}]
```

Checking Supported Tasks and Models

When using `transformers.pipeline()`, if you don't specify a `model`, it defaults to a suitable pre-trained model for the given `task` (e.g., `sshleifer/distilbart-cnn-12-6` for summarization). The `transformers` library handles the mapping of tasks to models automatically.

To 'check supported tasks' more broadly or to understand the capabilities of a specific model, you would typically:

1. **Consult the Hugging Face Model Hub:** Go to huggingface.co/models and filter by task (e.g., "summarization"). Each model card provides details on what tasks it supports, its architecture, and how to use it.
2. **Refer to `transformers` Documentation:** The official `transformers` documentation provides comprehensive guides on various tasks and how to use the `pipeline` for them.
3. **Inspect the `pipeline` signature:** While `pipeline` itself doesn't have a `list_supported_tasks()` method, understanding its parameters (like `task`, `model`, `tokenizer`) helps you select appropriate combinations. For example, if you explicitly set `pipeline("summarization", model="your/custom-model")`, you'd need to ensure 'your/custom-model' is indeed a summarization model.

In most cases, if `pipeline("summarization")` initializes successfully, the library has found a model capable of summarization, and any `KeyError` is more likely due to an incorrect assumption about the output structure rather than the model not supporting the task itself.

✓ Technical Phase (Implementation)

```
# Audit: Sentiment Analysis
test_sentences = [
    "I love this new product! It works amazingly well.",
    "The service was terrible and slow.",
    "This is okay, nothing special.",
    "Great job everyone, really appreciate it.",
    "Oh sure, because that always works perfectly."
]

for sentence in test_sentences:
    result = sentiment_pipeline(sentence)[0]
    print(f"Text: {sentence}")
    print(f"Sentiment: {result['label']} (score: {result['score']:.4f})")
    print("---")
```

```
Text: I love this new product! It works amazingly well.
Sentiment: POSITIVE (score: 0.9999)
---
Text: The service was terrible and slow.
Sentiment: NEGATIVE (score: 0.9996)
---
Text: This is okay, nothing special.
Sentiment: NEGATIVE (score: 0.9895)
---
Text: Great job everyone, really appreciate it.
Sentiment: POSITIVE (score: 0.9999)
---
Text: Oh sure, because that always works perfectly.
Sentiment: POSITIVE (score: 0.9999)
---
```

```
# Creator: Text Generation
prompt = "In the context of customer support, suggest ways to improve response time"
generated = generator(prompt, max_length=50, num_return_sequences=1, pad_token_id=5)
print("Generated text:")
print(generated)
```

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Write customer support emails

Note: You can also use customer support emails to communicate with your partner's

Read more about customer support emails in our product guide.

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support Emails

Write Customer Support

Executive Summary: Summarization

news_article = """

Tech companies are rapidly adopting AI for customer service. Recent studies show that freeing human agents for complex issues. However, challenges remain in understanding Major firms like Google and Microsoft are investing heavily in transformer-based models.

summary = summarizer(news_article, max_length=30, min_length=20, do_sample=False)[0]

```
print("Original length:", len(news_article.split()))  
print("Summary:", summary)
```

Original length: 57

Summary:

Tech companies are rapidly adopting AI for customer service. Recent studies show that freeing human agents for complex issues. However, challenges remain in understanding Major firms like Google and Microsoft are investing heavily in transformer-based mod

✓ Action Phase

Pre-training vs Fine-tuning:

Pre-training is like teaching a model the general rules of language by exposing it to massive amounts of text data. It learns patterns, grammar, facts about the world. Fine-tuning is then specializing that knowledgeable model for a particular task, like sentiment analysis, by training it on labeled examples for that specific job.

Business Workflow for Customer Support:

1. Incoming ticket -> Sentiment Analysis to prioritize urgent/negative cases.
2. Summarization to create quick executive summaries for supervisors.
3. Text Generation to draft initial responses or suggest solutions.
4. Human review for final approval and personalization.

This could reduce handling time for 1,000 tickets/day significantly.

Hallucination/Bias Example & Risks:

In generated text, models may invent facts. Over-reliance risks poor customer experience, spreading misinformation, or amplifying biases present in training data. Always have human oversight.